Multivariate analysis exam

UCT Statistics Honours 2026

Table of contents

Questions

Analysis I

```
if (!exists("read_exam_data", mode = "function")) {
  stop("Please run the first chunk to define the 'read_exam_data' function.")
}
data_tidy_stress <- read_exam_data("data_tidy_stress.csv")
```

```
Rows: 500 Columns: 6
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (6): age, income, satisfaction, experience, hours, stress

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Question 1.1

Part a

```
# View(data_tidy_stress)
# Calculate correlation matrix:
cor_mat = cor(data_tidy_stress)
cor_mat
```

```
                   age     income satisfaction experience       hours
age          1.00000000 0.83271764    0.6392036 0.96823135  0.06373114
income       0.83271764 1.00000000    0.7284207 0.80566405  0.06219594
satisfaction 0.63920359 0.72842066    1.0000000 0.62688896 -0.16060620
experience   0.96823135 0.80566405    0.6268890 1.00000000  0.06068512
hours        0.06373114 0.06219594   -0.1606062 0.06068512  1.00000000
stress       0.02257214 0.04061594   -0.2540621 0.01528969  0.73491605
                 stress
age           0.02257214
income        0.04061594
satisfaction -0.25406213
experience    0.01528969
hours         0.73491605
stress        1.00000000
```

Part b

```
pca_mod = prcomp(cor_mat)
pca_mod
```

```
Standard deviations (1, .., p=6):
[1] 1.088655e+00 2.613981e-01 1.210127e-01 9.650339e-02 1.372953e-02
[6] 1.818186e-17
```

```
Rotation (n x k) = (6 x 6):
                    PC1        PC2         PC3        PC4         PC5
age          -0.3848419  0.5172837 -0.06244625  0.1817455 -0.73982921
income       -0.3726551  0.1975545  0.13908190 -0.8615479  0.11037371
satisfaction -0.4570689 -0.4646920 -0.09989388 -0.1265417 -0.11575026
experience   -0.3826382  0.5306103 -0.08938969  0.2947537  0.64805073
hours         0.4083432  0.2675221 -0.77203521 -0.3110925 -0.03872624
stress        0.4369859  0.3526127  0.60228649 -0.1582172 -0.07485184
                    PC6
age          -0.006006763
income       -0.219793998
satisfaction  0.731956015
experience    0.239166702
hours         0.259559423
stress        0.539743553
```

Part c

Part d

Part e

Part f

Analysis II

```
if (!exists("read_exam_data", mode = "function")) {
  stop("Please run the first chunk to define the 'read_exam_data' function.")
}
img_list <- read_exam_data("pca_image_results.rds")
```

Question 2.1

Part a

No, PCA must be performed on standardised data or variables with high variance will dominate the principal components.

Part b

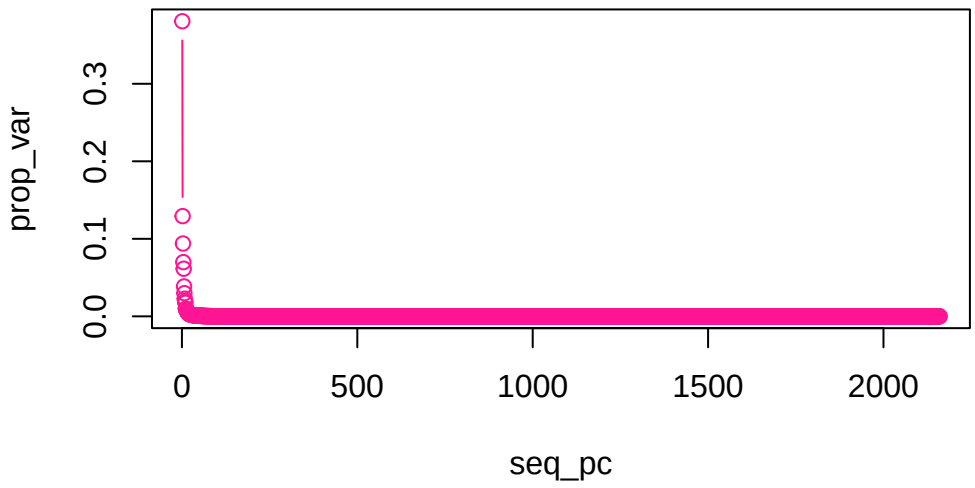
```
# View(img_list)
# Reconstruct original image:
scores = img_list$V
loadings = img_list$Y
eigenvalues = img_list$lambda
means = img_list$M

og_img = t(scores%*%t(loadings))+means
save_img(og_img,'reconstructed_image.jpg')
```

```
[1] "reconstructed_image.jpg"
```

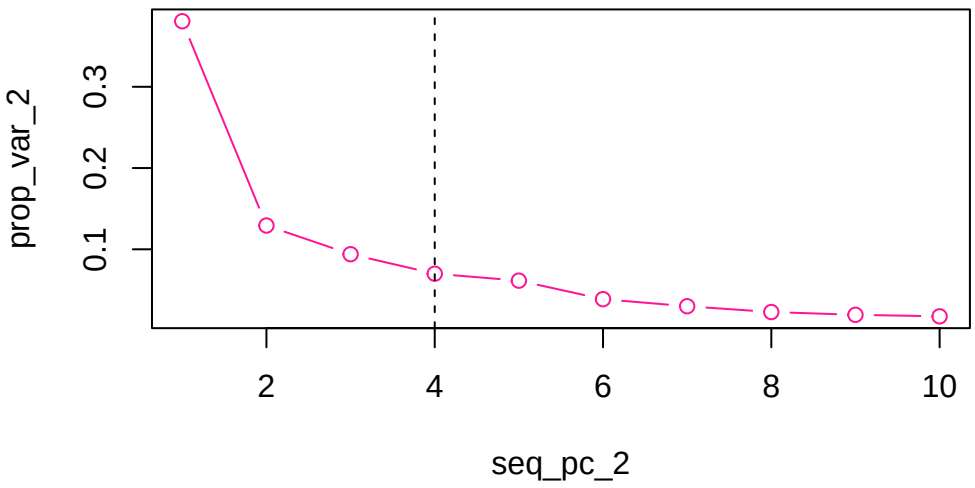
```
knitr::include_graphics('reconstructed_image.jpg')
```

Part c

```
# Determine optimal no. principal components:
# Total variance:
total_var = sum(eigenvalues)

# Proportion of variance explained:
prop_var = eigenvalues/total_var

# Principal components:
seq_pc = seq(1:length(eigenvalues))

# Plot proportion of variance explained by components:
plot(seq_pc,prop_var,
     type='b',
     col='deeppink')
```




```
# Proportion of variance explained:
# First 10 PCs
prop_var_2 = prop_var[1:10]
seq_pc_2 = seq(1:10)

# Plot proportion of variance explained by first 10 PCs:
plot(seq_pc_2,prop_var_2,
     type='b',
     col='deeppink')
abline(v=4,lty=2)
```




```
# Reconstruct image:
scores_2 = scores[]
loadings_2 = loadings[]
eigenvalues_2 = eigenvalues[]
means_2 = means[]

recon_img = t(scores_2%*%t(loadings_2))+means_2
save_img(recon_img,'reconstructed_image_2.jpg')
```

```
[1] "reconstructed_image_2.jpg"
```

```
knitr::include_graphics('reconstructed_image_2.jpg')
```

Analysis III

```
if (!exists("read_exam_data", mode = "function")) {
  stop("Please run the first chunk to define the 'read_exam_data' function.")
}
data_tidy_lum_response <- read_exam_data("data_tidy_lum_response.csv")
```

```
Rows: 100 Columns: 16
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): pid
dbl (15): crp, ddimer, fractalkine, ifng, il10, il1a, il1ra, mcp1, mip1a, mi...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data_tidy_lum_meta <- read_exam_data("data_tidy_lum_meta.csv")
```

```
Rows: 100 Columns: 6
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (6): pid, infxn, age, sex, ethnicity, household_size

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
data_tidy_lum <- data_tidy_lum_meta |>
  dplyr::left_join(data_tidy_lum_response, by = "pid") |>
  dplyr::mutate(
    age_infxn = paste0(age, "_", infxn)
  ) |>
  dplyr::select(pid:household_size, age_infxn, everything())
```

Question 3.1

Part a

```
library(ca)
# View(data_tidy_lum_meta)
# Create a row-principal plot:
# row = age
# col = ethnicity
```

Part b

Part c

Part d

Question 3.2

Part a

Part b

Part c

Part d

Question 3.3

In `data_tidy_lum_response`, the variables `crp` to `il1ra` are one set of variables ($\mathbf{X}^{(1)}$), and the variables `mcp1` to `scd40l` are another set ($\mathbf{X}^{(2)}$).

Standardise the variables in `data_tidy_lum_response`, creating a new matrix `lum_response_mat_std`, using this code, and use `lum_response_mat_std` for question 3.3:

```
if (!exists("data_tidy_lum")) {
  stop("Please run code above to create the `data_tidy_lum` object.")
}
lum_response_mat_std <- data_tidy_lum |>
  dplyr::select(crp:scd40l) |>
  dplyr::mutate(across(dplyr::everything(), ~ (.-mean(.))/sd(.))) |>
  as.matrix()
```

Part a

Part b

Part c

Question 3.4

Part a

```
# View(data_tidy_lum)
# Step 1:
dat = data_tidy_lum[,-(1:6)]
X = dat[,-1] # predictors
g = length(unique(dat$age_infxn)) # no. groups
n_i = table(dat$age_infxn)
```

```r
bar_overall = colMeans(X)
bar_group = dat|>
  dplyr::group_by(age_infxn)|>
  dplyr::summarise(
    across(everything(),mean),
    .groups='drop'
  )|>
  dplyr::select(-age_infxn)|>
  as.matrix()

#v Step 2:
B = Reduce(`+`,lapply(1:g,function(i){
  n_i[i]%*%(t(bar_group[i,]-bar_overall))%*%(bar_group[i,]-bar_overall)
}))

W = Reduce(`+`,lapply(1:g,function(i){
  Xi = X[dat$age_infxn==i,]
  cov(Xi)*(n_i[i]-1)
}))
```

Step 3:

eig = eigen(solve(W)%%B) *eig_vec =eig%vectors |> Re() S_pooled = W/(sum(n_i)-g) sd_vec = diag(1/S_pooled)|>sqrt() eig_vec = eig_vec*sd_vec eig_vec = eig_vec[,1:2]

Step 4:

scores = X%*%eg_vec dat$LD1 = scores[, 1]$dat$LD2 = scores[,2]

Step 5:

ggplot(data=dat, aes(x=LD1,y=LD2,color=age_fxn))+ geom_point()

Part b

Multiple choice

- 4.1: a
- 4.2: d
- 4.3: c
- 4.4: b
- 4.5: a
- 4.6: a
- 4.7:
- 4.8: